



Sukkur Institute of Business Administration University

Department of Computer Science

Object Oriented Programming using Java

BS – II (CS/AI/SE)

Spring-2025

Lab # 06: Let's learn about class Parameter passing, Recursion and Association basics

Instructor: Nimra Mughal

Objectives

After performing this lab, students will be able to understand:

- Pass by value, pass by reference
- Return Objects
- Recursion

Pass by Reference

```

/*In this example we have passed the object as a parameter*/
class Rectangle {
    int length;
    int width;

    Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }
}

public class PassObjectExample {
    public static void main(String[] args) {
        Rectangle rect = new Rectangle(5, 4);
        System.out.println("Before modifying: Length = " + rect.length + ", Width = " + rect.width);
        modifyRectangle(rect);
        System.out.println("After modifying: Length = " + rect.length + ", Width = " + rect.width);
    }

    public static void modifyRectangle(Rectangle r) {
        r.length = 10;
        r.width = 8;
    }
}

```

```

/*In this example, we have passed the array as an object which is another example
of pass-by reference*/
public class Example {
    public static void modifyArray(int[] arr) {
        arr[0] = 10;
    }

    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};

        modifyArray(numbers);

        System.out.println(numbers[0]); // Output: 10
    }
}

```

Return object

Implement a method to search for a product and return the product object containing details like name, price, and availability. Ensure proper handling of object return and manipulation.

```
import java.util.Scanner;

class Product {
    String name;
    double price;
    boolean available;

    Product(String name, double price, boolean available) {
        this.name = name;
        this.price = price;
        this.available = available;
    }

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Price: $" + price);
        System.out.println("Availability: " + (available ? "Available" : "Not
Available"));
    }
}

public class ShoppingCart {
    static Product[] products = new Product[3]; // Array to hold products

    static int cartSize = 0;

    public static void main(String[] args) {
        // Adding sample products
        products[0] = new Product("Laptop", 999.99, true);
        products[1] = new Product("Smartphone", 599.99, true);
        products[2] = new Product("Headphones", 99.99, false);

        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter product name to search: ");
        String searchName = scanner.nextLine();
        Product foundProduct = searchProductByName(searchName);
        if (foundProduct != null) {
            System.out.println("Product found:");
            foundProduct.displayInfo();
        }
    }
}
```

```

        System.out.println("Adding product to cart...");
        addToCart(foundProduct);
        System.out.println("Product added to cart successfully!");
    } else {
        System.out.println("Product with name '" + searchName + "' not
found.");
    }
}

public static Product searchProductByName(String name) {
    for (Product product : products) {
        if (product != null && product.name.equalsIgnoreCase(name)) {
            return product;
        }
    }
    return null; // Return null if product not found
}
}

```

Final keyword

Develop a Java program for a **banking system**. Use static variables and methods to keep track of the total number of accounts and the bank's interest rate. **Utilize final** keyword to create immutable classes for important entities like Account, ensuring their values cannot be changed once initialized.

```

class Account {
    private final int accountNumber;
    private final String accountHolderName;
    private double balance;

    public Account(int accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }

    public int getAccountNumber() {
        return accountNumber;
    }
}

```

```

    public String getAccountHolderName() {
        return accountHolderName;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println(amount + " deposited successfully. Current balance: " +
balance);
    }

    public void withdraw(double amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println(amount + " withdrawn successfully. Current balance:
" + balance);
        } else {
            System.out.println("Insufficient balance. Withdrawal failed.");
        }
    }
}

public class BankingSystem {
    private static int totalAccounts = 0;
    private static final double INTEREST_RATE = 0.05;

    public static void main(String[] args) {
        Account account1 = new Account(101, "Alice", 1000);
        Account account2 = new Account(102, "Bob", 2000);

        totalAccounts += 2; // Increment totalAccounts when new accounts are

        System.out.println("Total number of accounts: " + totalAccounts);
        System.out.println("Bank's interest rate: " + INTEREST_RATE);

        // Perform operations on accounts
        account1.deposit(500);
        account2.withdraw(300);
    }
}

```

Recursion

Base case (i.e., when to stop), where the function directly computes an answer without calling itself.

Recursive case (i.e., call ourselves), where the function calls itself as part of the computation.

Perhaps the simplest example is calculating factorial: $n! = n \cdot (n - 1) \cdot \dots \cdot 2 \cdot 1$. However, we can also see that $n! = n \cdot (n - 1)!$. Thus, factorial is defined in terms of itself. For example,

$$\begin{aligned}
 \text{factorial}(5) &= 5 * \text{factorial}(4) \\
 &= 5 * (4 * \text{factorial}(3)) \\
 &= 5 * (4 * (3 * \text{factorial}(2))) \\
 &= 5 * (4 * (3 * (2 * \text{factorial}(1)))) \\
 &= 5 * (4 * (3 * (2 * (1 * \text{factorial}(0))))) \\
 &= 5 * (4 * (3 * (2 * (1 * 1)))) \\
 &= 5 * 4 * 3 * 2 * 1 * 1 = 120
 \end{aligned}$$

We can trace this computation in precisely the same way that we trace any sequence of function calls.

```

factorial(5)
  factorial(4)
    factorial(3)
      factorial(2)
        factorial(1)
          return 1
        return 2*1 = 2
      return 3*2 = 6
    return 4*6 = 24
  return 5*24 = 120

```

When a recursive call is made, new storage locations for variables are allocated on the stack

Exercises

Mega Exercise: Solve Exercises from the HackerRank (OOP track topics). As many as you can. I have listed here two basic challenges)

<https://www.hackerrank.com/domains/java>

1. Exercise 1 Pass by reference

MedicalInventoryManager

Create a Java program to manage medical inventory. The program should include two classes namely **Medicine** and **MedicalInventoryManager**. The medicine class has **MedicineName, strength, and Quantity** attributes and a Method `displayDetails()` to display the details of medicines. **MedicalInventoryManager** class has a method to update the quantity of a particular medicine by passing its reference and to display the details of a medicine by passing its value.

Method Names:

`updateMedicineQuantity(Medicine medicine, int quantityToAdd)`

This method updates the quantity of a particular medicine by passing its reference.

Parameters:

medicine: Reference to the medicine object whose quantity needs to be updated.

quantityToAdd: The quantity to add to the current quantity of the medicine.

`displayDetails()`

This method displays the details of a medicine by passing its value.

2. Exercise 2 (Return object)

In previous classes, you implemented a program where you created an array of Students. Please extend that code, Implement a method to search for a student by ID, and return the student object.

- **Student** class represents a student with attributes such as id, name, and age.
- **StudentDatabase** class holds the main method. It also contains a static array **students**

to store student objects.

- In the **main** method, **students** array is created and values are initialized for each student.
- A scanner object is created to get user input for searching a student by ID.
- The **searchStudentById(int ID)** takes the **ID** as a parameter and iterates through the **students** array to find a student with the given ID. If found, it returns the student object; otherwise, it returns null.
- The result of the search is displayed in the **main** method. If the student is found, their information is displayed; otherwise, a message indicating that the student was not found is displayed.

3. Exercise 3 Even Fibonacci Sum (Recursion) EvenFibSum.java

Write a method that returns the sum of all Fibonacci numbers using recursion. Consider all Fibonacci numbers that are less than or equal to n.

Each new element in the Fibonacci sequence is generated by adding the previous two elements.

1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

Method should be:

```
public Integer evenFibonacciSum(Integer n)
```

4. Exercise 4 (Recursion)

Write a recursive function to compute power of a number (X^n). Test and trace for 4 5 ? Hint:

$4^5 = 4 * 4^4$; $4^0 = 1$.